

TRABAJO FINAL INTEGRADOR

Alumno: Jorge Enrique NICOLAU

E-mail: jenicolau@gmail.com

Solución Propuesta: Central de Control Domótico

Empresa o industria a aplicar: Control electrónico de dispositivos caseros.

Objetivo del trabajo: Describir una solución de uso doméstico, sus componentes, tecnología, mapa de solución e implementación a alto nivel. La plataforma de control domótico de dispositivos de iluminación contará con administración vía Web y vía móvil tanto para monitoreo de sensores como para utilización de activadores; además contará con un módulo de programación de artefactos y simulación de presencia.

Ámbito de aplicación de la solución WiFi y Ethernet: Control Domótico en base a la información recolectada por sensores ambientales y Simulación de Presencia.

Descripción de componentes de la solución

Sensores a utilizar:

- **Temperatura y Humedad: Sensor DHT22¹.** De los sensores más comunes de uso más extendido con Arduino están los sensores DHT11 y DHT22²; se eligió este último por su mejor rango de medición tanto de temperatura (0 a 50 °C el DHT11 vs -40 a 80 °C el DHT22) como de humedad (20 a 90% del DHT11 vs 0 a 100% del DHT22); además tiene la ventaja de consumir menos energía ya que el DHT22 envía lecturas cada 2 segundos contra 1 segundo del DHT11. Por último, para un nivel de precio no muy distinto (USD 1 a 5 del DHT11 vs USD 4 a 10 DHT22) el DHT22 (+/-0.5°C y +/-2% de humedad) es más preciso que DHT11 (+/-2 °C y +/-5% de humedad)
- **Fotoeléctrico: Sensor basado en LDR y comparador LM393³.** Generalmente implementado como un módulo⁴ de detección de un fotorresistor y comparador de voltaje en un chip para conexión a Arduino.

Actuadores a utilizar:

- **Relés: Módulo con relé SONGLE⁵.** Normalmente implementado en un módulo de un canal a 5V compatible con el conexionado de Arduino⁶, se selecciona este módulo dada su capacidad de carga a controlar (hasta 10A 250VAC).

Protocolos a utilizar:

- MQTT (conexión de dispositivos IoT con Broker),
- HTTP (conexión de interfaz de usuario final con Node-RED para composición de Dashboard),

¹ Digital-output relative humidity & temperature sensor/module DHT22 (DHT22 also named as AM2302) Datasheet <https://www.sparkfun.com/datasheets/Sensors/Temperature/DHT22.pdf>

² Complete Guide for DHT11/DHT22 Humidity and Temperature Sensor With Arduino <https://randomnerdtutorials.com/complete-guide-for-dht11-dht22-humidity-and-temperature-sensor-with-arduino/>

³ LMx93-N, LM2903-N Low-Power, Low-Offset Voltage, Dual Comparators <http://www.ti.com/lit/ds/symlink/lm2903-n.pdf>

⁴ Photoresistor sensor module light detector for Arduino <https://www.buildcircuit.com.au/photoresistor-sensor-module-light-detector-for-arduino-in-sydney-australia/>

⁵ SONGLE Relay <http://www.circuitbasics.com/wp-content/uploads/2015/11/SRD-05VDC-SL-C-Datasheet.pdf>

⁶ Modulo Relé 1 Canal 5V para Arduino <https://www.mrwatt.eu/es/tienda/electronica-y-domotica/electronica-y-domotica-modulos/modulo-rele-1-canal-5v-para-arduino.html>

- WiFi (conexión de placas inalámbricas con Broker MQTT)
- Ethernet (conexión de placas cableadas con Broker MQTT).

Gateway MQTT: Broker en la red local implementado con un **RaspberryPi Zero W**⁷ tanto con interfaz Ethernet (OTG Ethernet⁸) como WiFi (onboard en el Pi Zero W). El software de broker se implementará con una instalación de **Mosquitto**⁹ **MQTT Server**.

Interfaces Gráficas y Dashboards: Dashboard se implementará con una instalación local del **Node-RED Dashboard**¹⁰, también funcionando en el RaspberryPi Zero W.

Descripción Funcional General de la solución

La solución consiste en una central domótica orientada al control de la iluminación y simulación de presencia. Consiste en dos tipos básicos de transductores:

1. Los actuadores son placas con módulo relé compatible con Arduino conectado a una placa NodeMCU ESP8266¹¹ (específicamente el board **ESP8266 12E NodeMCU**¹² basado en el MCU **ESP8266**) si son inalámbricos o uno **Arduino Uno**¹³ con **shield Ethernet**¹⁴ si están "cableados" a la LAN.
2. Los sensores son placas con módulo de sensor fotoeléctrico compatible con Arduino conectado a una placa Arduino Nano conectado a una placa NodeMCU si son inalámbricos o uno Arduino Uno con shield Ethernet si están conectados a la LAN. También hay una versión que conecta un sensor de temperatura y humedad a la placa Arduino o NodeMCU.

Los dos tipos de placa tiene en el microcontrolador una rutina que implementa un cliente de MQTT (ver sección de conectividad detallada).

Para el caso de la placa actuadora está leyendo a intervalos regulares su correspondiente tópico de estado en el broker MQTT y reacciona abriendo o cerrando el relé en función del valor leído.

Para el caso de la placa con el sensor está escribiendo a intervalos regulares sus lecturas del sensor fotovoltaico o de valores de temperatura y de humedad ambiente en su correspondiente tópico en el broker MQTT.

Por ser dispositivos hogareños, la alimentación de las placas está dadas por fuentes adaptadoras de 220V CA a 5V CC.

La "central" de este esquema domótico está implementado por una Raspberry Pi Zero W que implementa un Broker MQTT, un servidor Node-RED y un backend con cliente MQTT (incorporado en Node-RED). La parte del servicio del broker del gateway Raspberry es el que se encarga de publicar en los tópicos de los sensores y distribuir en los tópicos de los actuadores. La parte del servidor Node-RED publica una aplicación web responsiva de modo de poder ser accedida tanto por clientes PC como clientes móviles con una interfaz de usuario adaptativa.

La parte de los servicios de backend sobre los que la interfaz Web lee o escribe datos implementa un cliente MQTT que se comunica con los tópicos a monitorear o controlar; toda esta interfaz está implementada con Node-RED.

Un caso tipo de interacción sensor/actuador es el sensor de humedad reportado el valor en tópico en el que también esté suscrito un actuador de relé que tenga conectado un humidificador. La relación de en cuál umbral se dispara la activación del humidificador, por ejemplo, se implementa en el flujo de

⁷ Raspberry Pi Zero W <https://www.raspberrypi.org/products/raspberry-pi-zero-w/>

⁸ Plugable USB 2.0 OTG micro-b to 10/100 ethernet adapter <https://plugable.com/products/usb2-otge100/>

⁹ Eclipse Mosquitto. An open source MQTT broker <https://mosquitto.org/>

¹⁰ node-red-dashboard 2.18.0. A set of dashboard nodes for Node-RED <https://flows.nodered.org/node/node-red-dashboard>

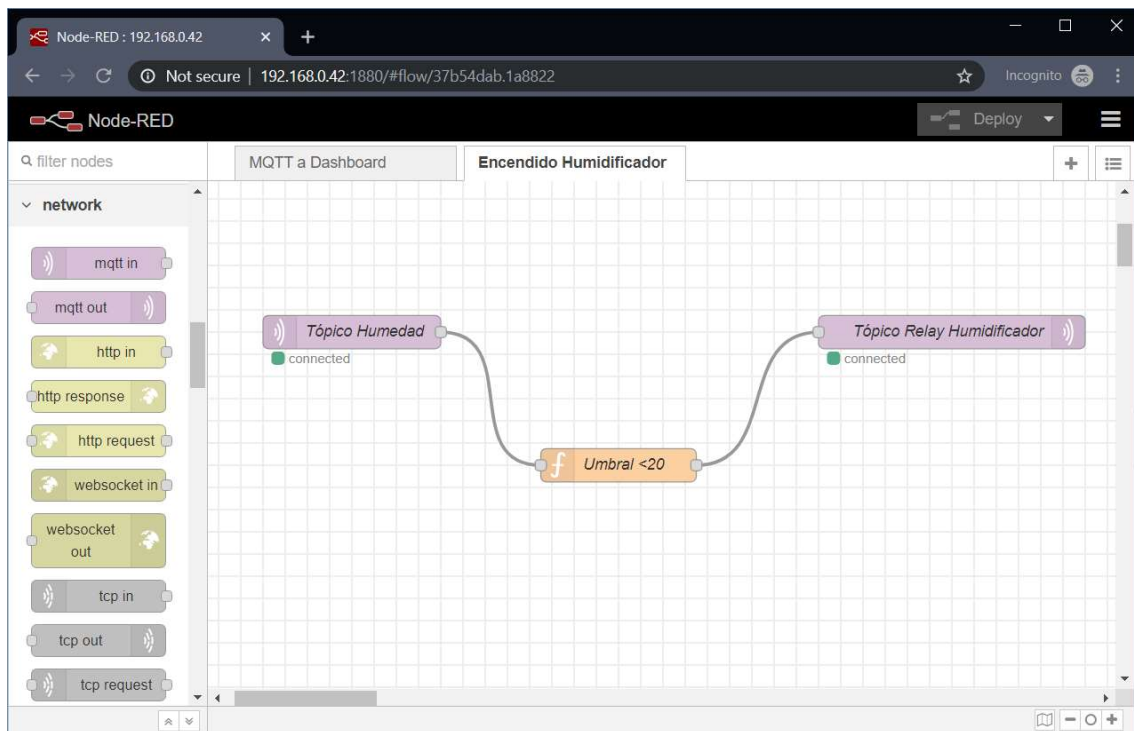
¹¹ NodeMcu: An open-source firmware and development kit that helps you to prototype your IOT product within a few Lua script lines https://www.nodemcu.com/index_en.html

¹² Insight Into ESP8266 NodeMCU <https://lastminuteengineers.com/esp8266-nodemcu-arduino-tutorial/>

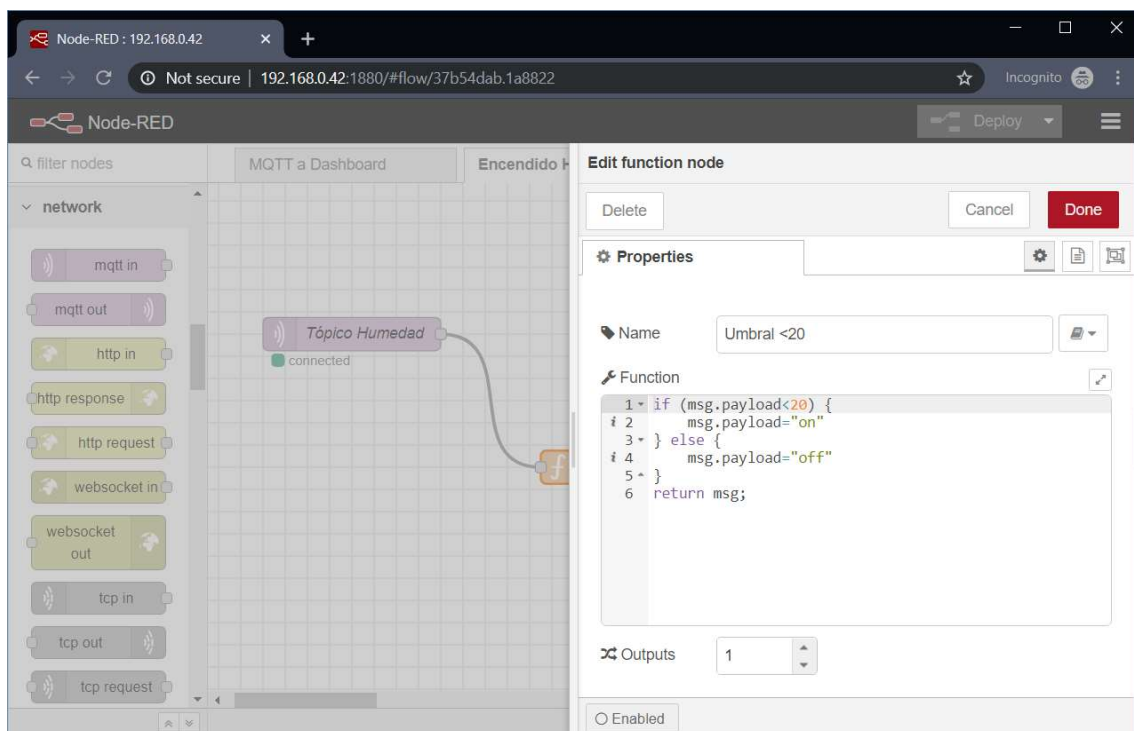
¹³ Arduino Uno Rev3 <https://store.arduino.cc/usa/arduino-uno-rev3>

¹⁴ Arduino Ethernet Shield 2 <https://store.arduino.cc/usa/arduino-ethernet-shield-2>

Node-RED que relación la lectura de un nodo **“network/mqtt in”**, un nodo **“function/function”** con la evaluación del umbral y un nodo **“network/mqtt out”** de la siguiente manera:

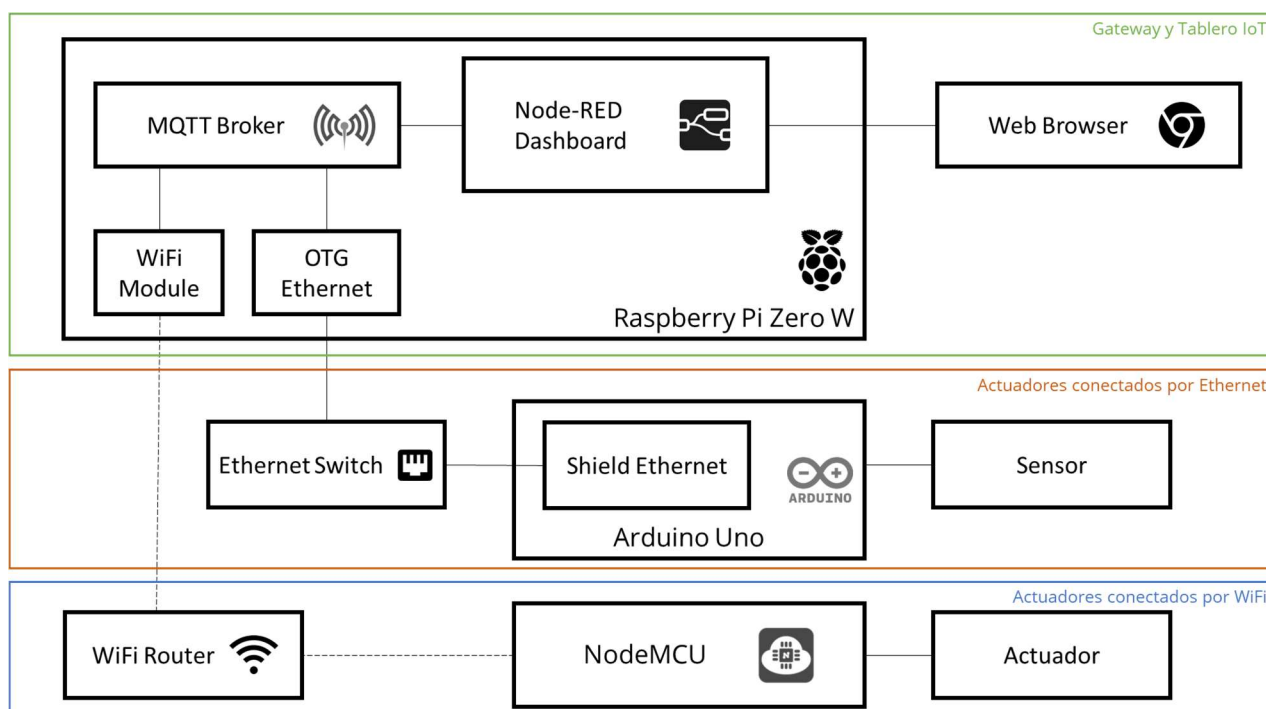


Y dentro de la función simplemente se evalúa si activar o no el humidificador (el valor en **msg.payload** es lo que publicará el nodo **“network/mqtt out”**):



Esquemático de Arquitectura General la Solución

El diagrama general de despliegue de la solución, simplificado a un actuador inalámbrico, un sensor conectado por LAN, el broker, las interfaces de usuario y el motor de reglas muestra cómo quedaría con la interconexión de componentes.



Así pueden diferenciarse en la Solución planteada tres sectores diferenciados:

- **Los actuadores conectados al Gateway por WiFi.** Son aquellos que implementan la pila tecnológica basada en NodeMCU ESP8266
- **Los actuadores conectados al Gateway por Ethernet.** Son aquellos que implementa la pila tecnológica basa en Arduino Uno con Shield Ethernet
- **Gateway y Tablero IoT.** Es la parte que implementa el broker MQTT local (con Mosquitto) y el server que va a interactuar con los tópicos MQTT (flujos de Node-RED) y va a generar la visualización del dashboard (con Node-RED Dashboard).

Ejemplo de Arquitectura Detallada de la Solución

En esta sección se describe el esquema total global con detalle de bajo nivel para la conexión desde el sensor a medir hasta la integración final en el dashboard. Para cada actuador que se vaya a implementar como parte de la solución debe implementarse un circuito similar al presentado en esta sección de arquitectura detallada variando principalmente y en función de cada caso:

- **Conexión del sensor o del actuador con la placa controladora.** Cada transductor tiene su particularidad de conexión y depende de la especificación del datasheet particular la forma en que se va a conectar físicamente a la placa controladora
- **Rutina de la placa controladora para la lectura espaciada de sensores o la escritura para activación de actuadores.** Si bien la solución de conexión física intenta simplificar el conexionado para generalmente leer o escribir información digital al adaptador es posible que también debe implementarse una rutina específica de lecto escritura analógica, PWM, transmisión serial entre placas, etc.
- **Rutina de la placa controladora para el correcto protocolo de acceso al medio.** En el caso de NodeMCU, son las bibliotecas de funciones para conectividad con WiFi; en el caso de Arduino con Shield Ethernet, son las bibliotecas para conectividad con Ethernet.
- **Rutina de la placa controladora que implemente el cliente MQTT adecuado.** Para el caso de la solución se trata de un cliente MQTT en la placa que sea capaz de leer desde o publicar hacia tópicos en el Mosquitto MQTT Broker¹⁵. Pero si se utilizase otra plataforma para la lectura

¹⁵ Arduino Client for MQTT <https://pubsubclient.knolleary.net/>

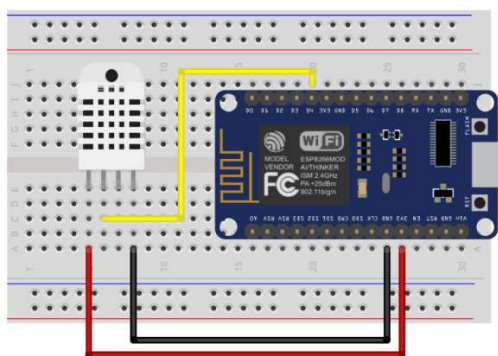
o escritura de datos, aquí iría implementada la rutina adecuada (por ejemplo, biblioteca Adafruit¹⁶ o cliente compatible con Cayenne¹⁷ o con Amazon IoT¹⁸).

- **Implantación del Broker MQTT para la solución.** En el caso de la solución presentada se utiliza Mosquitto MQTT Broker, pero podría reemplazarse con otra plataforma local o con una plataforma IoT en la nube (Amazon AWS IoT¹⁹, Google Cloud IoT²⁰, Azure IoT Hub²¹, etc.).
- **Implantación del dashboard para la solución.** En el caso presente se utiliza Node-RED²² para implementar el flujo de lectura o publicación de tópicos de control y visualizar los estados escritos o leídos mediante una UI Web que se incorpora como módulo adicional llamada Node-RED Dashboard. Si se elige una plataforma en la nube, en general estas proveen la capacidad de generar Dashboard con diseño responsivo (para usar tanto desde un Web Browser como de un smartphone) a partir de los tópicos generados.

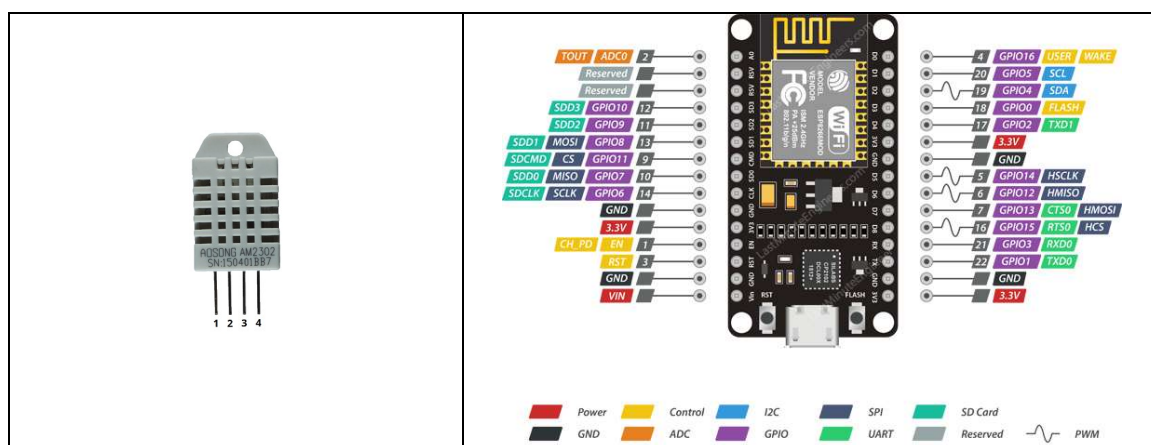
A continuación, y para explicar el circuito completo se va a tomar el caso del sensor de temperatura DHT22 conectado a una placa inalámbrica NodeMCU.

1. Conexión del Sensor DHT22 al NodeMCU ESP8266

El conexionado físico del sensor al MCU sigue el siguiente detalle:



Según el datasheet del DHT22 y del NodeMCU ESP8266 (ESP 12) la correlación es la siguiente:



¹⁶ Adafruit MQTT Library https://github.com/adafruit/Adafruit_MQTT_Library

¹⁷ Cayenne MQTT Arduino Library <https://github.com/myDevicesIoT/Cayenne-MQTT-Arduino>

¹⁸ AWS IoT Arduino Yún SDK <https://github.com/aws/aws-iot-device-sdk-arduino-yun>

¹⁹ AWS IoT. IoT services for industrial, consumer, and commercial solutions <https://aws.amazon.com/iot/>

²⁰ Google Cloud IoT. Unlock business insights from your global device network with an intelligent IoT platform <https://cloud.google.com/solutions/iot/>

²¹ Azure IoT Hub. Managed service to enable bi-directional communication between IoT devices and Azure <https://azure.microsoft.com/en-us/services/iot-hub/>

²² Node-RED. Low-code programming for event-driven applications <https://nodered.org/>

Pin DHT22 ²³	Pin ESP12-E ²⁴
VDD---power supply: 1	3V3: Los pines de 3.3V son salidas de un regulador de voltaje en la placa y pueden usarse para alimentar componentes externos.
DATA--signal: 2	17-GPIO2-TXD1: Cada GPIO puede utilizarse para entrada de datos digital
GND: 4	GND: Toma a tierra

Como se mencionó antes, este módulo NodeMCU está alimentado por una fuente adaptadora de 220V CA a 5V CC conectada al puerto micro USB del ESP12-E.

2. Rutina del NodeMCU para la lectura del DHT22, acceso al WiFi y cliente MQTT.

Para la lectura del sensor de temperatura y humedad, acceder al módulo WiFi del NodeMCU y escribir en el tópico correspondiente se utilizará la siguiente rutina:

```
// Rutina MCU para ESP8266, lectura DHT22 y escritura con MQTT

#include "DHT.h" // Bibliotecas con funciones de acceso al sensor DHT22
#include <PubSubClient.h> // Bibliotecas con funciones que implementan MQTT
#include <ESP8266WiFi.h> // Bibliotecas con funciones de conexión a WiFi

const char* ssid = "*****"; // En la implementación, el nombre de la WiFi
const char* password = "*****"; // En la implementación, el password de la WiFi

char* topicTemp = "/casa/temperatura"; // Tópico en el que va a escribir la temperatura
char* topicHum = "/casa/humedad"; // Tópico en el que va a escribir la temperatura
char* server = "XX.XX.XX.XX"; // Server MQTT--aquí va la IP del Raspberry Pi Zero W
char* hellotopic = "hello_topic"; //Tópico de Prueba

#define DHTPIN 17 // En cuál PIN está conectado
#define DHTTYPE DHT22 // DHT 22 (AM2302); código interno de DHT
#define REPORT_INTERVAL 30 // reporta la temperatura cada 30 segundos

String clientName;
DHT dht(DHTPIN, DHTTYPE, 15);
WiFiClient wifiClient;
PubSubClient client(server, 1883, callback, wifiClient);

float oldH ;
float oldT ;

void setup() {
  Serial.begin(38400);
  Serial.println("Inicialización DHT");
  delay(20);

  Serial.println();
  Serial.println();
  Serial.print("Conectando a ");
  Serial.println(ssid);

  WiFi.mode(WIFI_STA);
  WiFi.begin(ssid, password);

  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  Serial.println("");
  Serial.println("WiFi Conectado");
  Serial.println("Dirección IP: ");
  Serial.println(WiFi.localIP());

  clientName += "esp8266-";
  uint8_t mac[6];
  WiFi.macAddress(mac);
  clientName += macToStr(mac);
  clientName += "-";
  clientName += String(micros() & 0xff, 16);
```

²³ Digital-output relative humidity & temperature sensor/module DHT22 (DHT22 also named as AM2302) Datasheet <https://www.sparkfun.com/datasheets/Sensors/Temperature/DHT22.pdf>

²⁴ ESP8266EX Datasheet Version 5.8 <https://lastminuteengineers.com/datasheets/ESP8266-datasheet.pdf>

```

Serial.print("Conectando a ");
Serial.print(server);
Serial.print(" como ");
Serial.println(clientName);

if (client.connect((char*) clientName.c_str())) {
    Serial.println("Conectado al Broker MQTT");
    Serial.print("El t pico es: ");
    Serial.println(topic);

    if (client.publish(hellotopic, "hello from ESP8266")) {
        Serial.println("Publicado OK");
    }
    else {
        Serial.println("Fall  Publicaci n");
    }
}
else {
    Serial.println("Fall  conexi n a MQTT");
    Serial.println("Se resetea y prueba de nuevo...");
    abort();
}

dht.begin();
oldH = -1;
oldT = -1;
}

void loop() {

    float h = dht.readHumidity();
    float t = dht.readTemperature();

    if (isnan(h) || isnan(t)) {
        Serial.println(" Falla al leer del sensor DHT!");
        return;
    }

    Serial.print("Humedad: ");
    Serial.print(h);
    Serial.print(" %\t");
    Serial.print("Temperatura: ");
    Serial.print(t);
    Serial.println(" *C ");

    String payloadH = h;
    String payloadT = t;

    if (t != oldT || h != oldH )
    {
        sendTemperature(payloadH,payloadT);
        oldT = t;
        oldH = h;
    }

    int cnt = REPORT_INTERVAL;

    while (cnt-->0)
        delay(1000);
}

void sendTemperature(String pLHum, String pLTem) {
    if (!client.connected()) {
        if (client.connect((char*) clientName.c_str())) {
            Serial.println("Conectado nuevamente a Broker MQTT");
            Serial.print("Los t picos son: ");
            Serial.print(topicTemp);
            Serial.print(" y ");
            Serial.println(topicHum);
        }
        else {
            Serial.println("Fall  conexi n a MQTT");
            Serial.println("Se resetea y prueba de nuevo...");
            abort();
        }
    }

    if (client.connected()) {
        Serial.print("Enviando lectura al server: Humedad ");
        Serial.println(pLHum);

        if (client.publish(topicHum, (char*) pLHum.c_str())) {
            Serial.println("Publicada Humedad OK");
        }
        else {
            Serial.println("Fall  publicaci n Humedad");
        }

        Serial.print("Enviando lectura al server: Temperatura ");
        Serial.println(pLTem);
    }
}

```



```

    if (client.publish(topicTemp, (char*) pLTem.c_str())) {
        Serial.println("Publicada Temperatura OK");
    }
    else {
        Serial.println("Falló publicación Temperatura");
    }
}

}

void callback(char* topic, byte* payload, unsigned int length) {
    // handle message arrived
}

String macToStr(const uint8_t* mac)
{
    String result;
    for (int i = 0; i < 6; ++i) {
        result += String(mac[i], 16);
        if (i < 5)
            result += ':';
    }
    return result;
}

```

3. Implantación del Mosquitto MQTT en el Raspberry Pi Zero W

El server Raspberry Pi debe automatizarse la inicialización de Mosquitto en las rutinas de inicio de Rapsbian. Para comprobar el funcionamiento de Mosquitto vamos iniciar una ventana con conexión SSH al RaspberryPi ZeroW con la suscripción al tópico de prueba con el comando:

```
mosquitto_sub -d -t hello_topic
```

En una nueva ventana con conexión SSH, similar al código cargado en el NodeMCU, se puede comprobar la instalación de mosquito publicando un mensaje al tópico de prueba (el "hello_topic") con el comando:

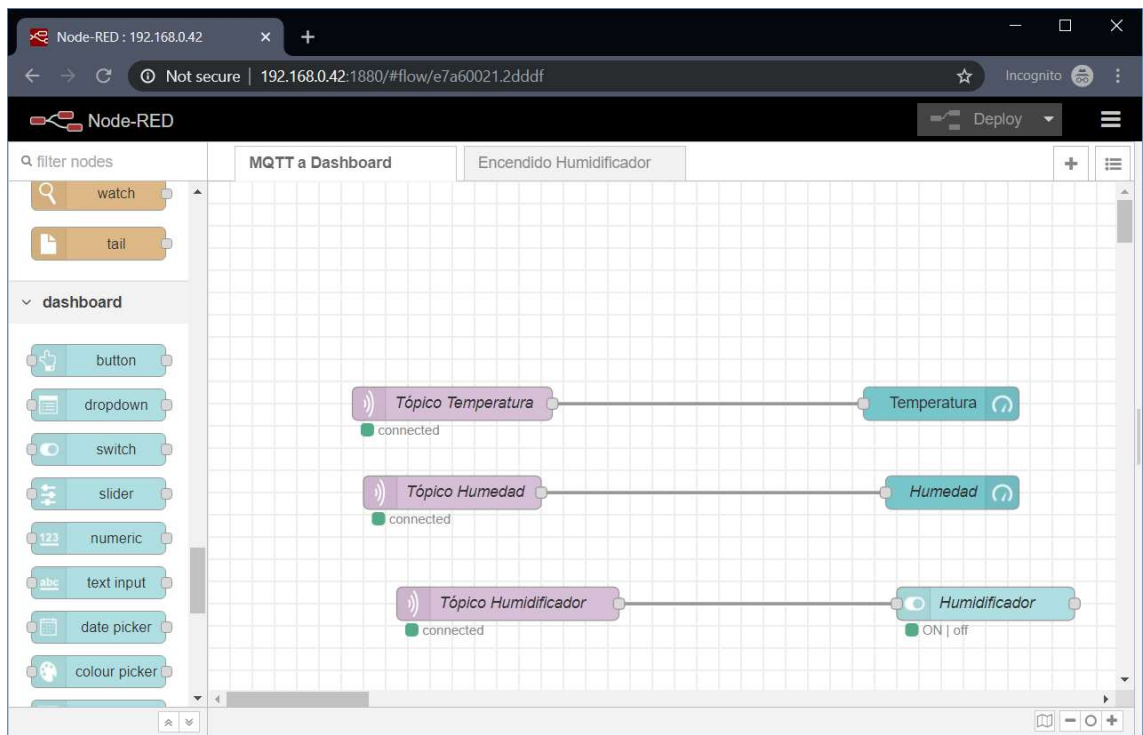
```
mosquitto_pub -d -t hello_topic -m "hello from RaspberryPi"
```

En la primera Ventana debería verse el mensaje "hello from RaspeberryPi" en el tópico al que se está suscripto (**hello_topic**).

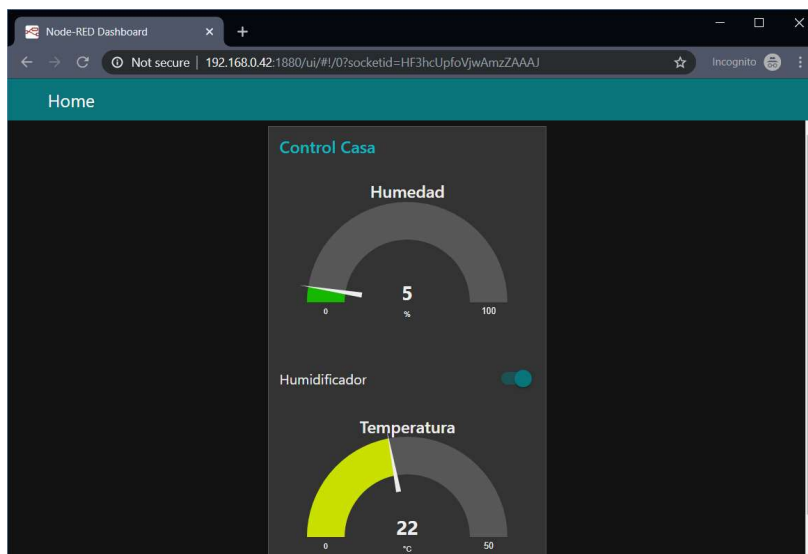
4. Implantación del dashboard en Node-RED en el RaspberryPi

En el caso presente se utiliza Node-RED para implementar el flujo de lectura o publicación de tópicos de control y visualizar los estados escritos o leídos. Node-RED es un motor de flujos con enfoque IoT, que permite definir gráficamente flujos de servicios, a través de protocolos estándares como REST, MQTT, Websocket, AMQP, etc. además de ofrecer integración con APIs de terceros, tales como Twitter, Facebook, Yahoo! y otros. Se trata de una herramienta visual, programada en NodeJS y que puede ejecutarse desde en dispositivos tan limitados como una Raspberry, hasta en plataformas complejas como AWS IoT o Azure IoT. Node-RED tiene, además, una extensión para crear y visualizar dashboards para interactuar con los dispositivos IoT que es la que se utilizará en esta solución.

Así, lo primero para visualizar el valor de un tópico MQTT en un elemento de un dashboard en Node-RED hay que generar el flujo que permita este manejo de datos, así definimos el flujo de lectura de los tópicos **"/casa/temperatura"** y **"/casa/humedad"** (e incluso muestra el estado de activación de **"/casa/humidificador"**) de nuestro MQTT Broker local:



Así conectamos dos nodos “**network/mqtt in**” con la debida parametrización de Broker y nombres de tópicos a dos nodos “**dashboard/gauge**”, los desplegamos haciendo click en “Deploy” e inmediatamente el dashboard estará disponible para la visualización (con interfaz responsiva), con actualización en tiempo real de los tópicos suscriptos:



Dashboard accedido desde una PC con un Web Browser



Dashboard accedido desde un Smartphone

Observaciones, conclusiones y necesidad de mejora

Esta solución está planteada como una prueba de concepto de la instalación punta a punta de todos los componentes de una implementación de IoT: desde el sensor hasta la visualización del usuario final. Se optó por no usar ninguna plataforma de IoT en la nube de modo de no ocultar ni delegar ninguno de los componentes (incluido broker MQTT y generar de dashboard).

Para una aplicación comercial, o al menos la versión “productiva” de la solución aquí desarrollada, la cantidad de componentes redundantes necesarios para hacer la instalación medianamente confiable hace que ya no tenga sentido utilizar todo instalado a mano y sin dependencias. Por ejemplo, es mucho más práctico utilizar un broker MQTT en la nube lo mismo que una plataforma de recolección de datos y generación de tableros/dashboards.

Un punto importante es la disponibilidad de la solución; si no se utilizasen plataformas en la nube, es necesario poner en funcionamiento todo un mecanismo para exponer el RaspberryPi Zero W a la Internet y eso puede llevar muchas tareas para que sea logrado con un grado razonable de seguridad.

La conexión física de transductores y componentes está planteada aquí en modo “hobbista” y utilizando breadboards para el montaje de prototipos temporales. En una versión de prueba habría que pensar en mandar a producir los módulos en placas preparadas para minimizar el empaquetamiento de cada uno de los dispositivos IoT. El uso de diseño adecuados a cada aplicación también puede llevar a placas optimizadas en el uso de la energía, y puede plantearse el despliegue de la solución con dispositivos autónomos de la red eléctrica, esto es alimentados a batería. La incorporación del uso de batería haría necesario también que estos dispositivos reporten el nivel de batería para planificar el mantenimiento preventivo de cada uno.

Finalmente, la solución de domótica aquí presentada no se presta para generar un módulo de explotación de datos interesante, pero si puede pensarse en un módulo histórico de almacenamiento de los datos recolectados que permita elaborar perfiles históricos de la medición con herramientas de Data Science.